

Temporal Logics: LTL, CTL, CTL*

Extension of CTL: $CTL^* = BTTL$

- In CTL, each linear-time operators G , F , X , and U must be immediately preceded by a path quantifier
- CTL^* extends CTL such that a path quantifier can prefix an assertion composed of
 - *arbitrary combinations of the usual linear-time operators (F , G , X and U)*
- Example:
 - EFp is a basic modality of CTL;
 - $E(Fp \wedge Fq)$ is a basic modality of CTL^*

Syntax of CTL*

- Two classes of formulas in *Backus Naur Form*
 - State formulas: evaluated in states
 - $f ::= p \mid \text{True} \mid (\neg f) \mid (f \wedge f) \mid A[a] \mid E[a]$, where p is any atomic formula and a any path formula;
 - Path formulas: evaluated along paths
 - $a ::= f \mid (\neg a) \mid (a \wedge a) \mid (a \cup a) \mid (Ga) \mid (Fa) \mid (Xa)$, where f is any state formula.
- An inductive definition is mutually recursive
 - the definition of each class depends upon the definition of the other.

Formulas in CTL*

- $A[(p \cup r) \vee (q \cup r)]$
 - Along all paths, either p is true until r , or q is true until r .
 - It is not equivalent to: $A[(p \vee q) \cup r]$.
 - Cannot be expressed in CTL.

Formulas in CTL*

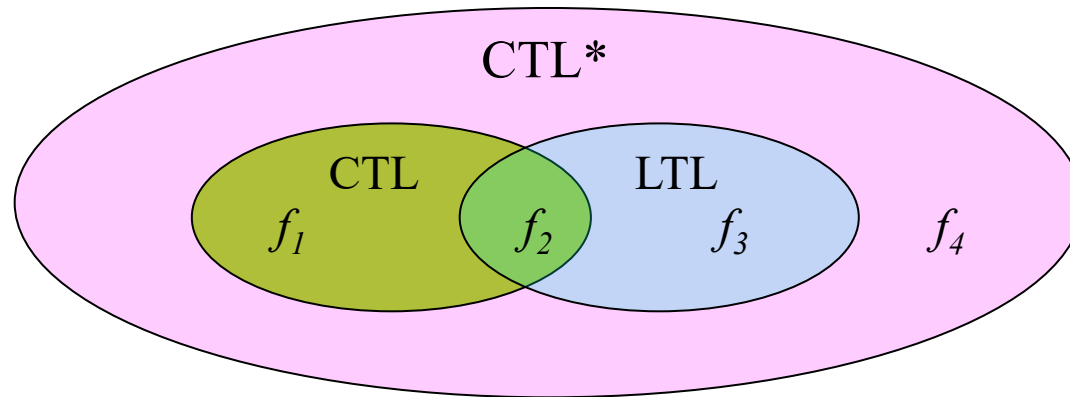
- $E[Xp \vee XX p]$
 - It is not equivalent to: $EXp \vee EXEX p$.
 - Cannot be expressed in CTL.
- $E[GF p]$
 - There is a path along which p is infinitely often true.
 - It is not equivalent to: $EG EF p$.
 - Cannot be expressed in CTL.

Linear vs. Branching Time TL

- LTL:
 - Easier inclusion of fairness constraints
 - $AG\ EF\ p$ cannot be expressed;
 - advantages in compositional verification
- CTL:
 - fairness properties $GF\ p \Rightarrow GF\ q$ not expressible;
 - fairness constraints often specified using exception conditions H :
 - computation paths along which states satisfy every $!(H)$ infinitely often.

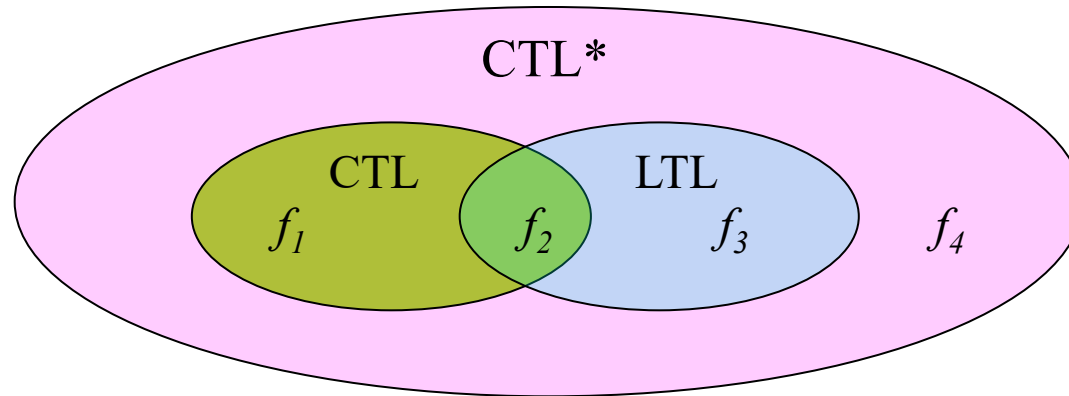
Linear vs. Branching Time TL

- CTL vs LTL:
 - Linear time conceptually simpler (words vs trees)
 - Branching time computationally more efficient



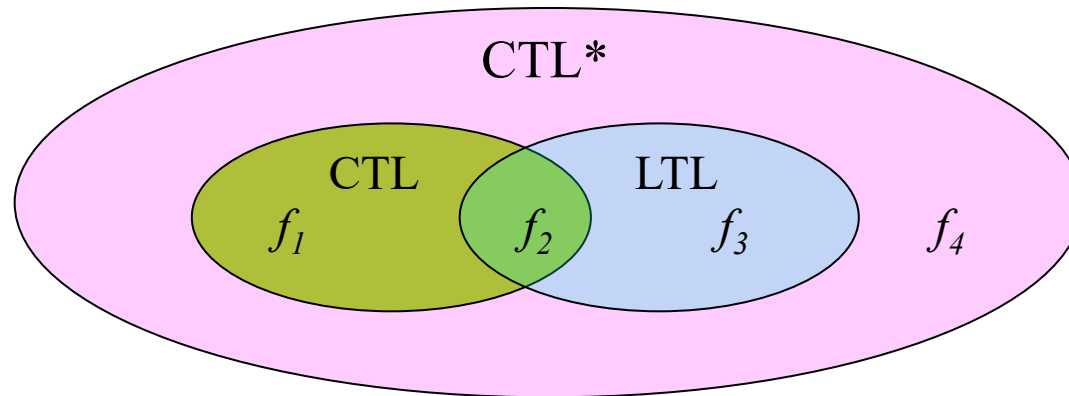
CTL-LTL $\neq\emptyset$

- In CTL, but not in LTL
 - $f_1 = AG\ EF\ p$
 - Wherever we have got to, we can always get back to a state in which p is true.
 - Useful in finding deadlocks in protocols.



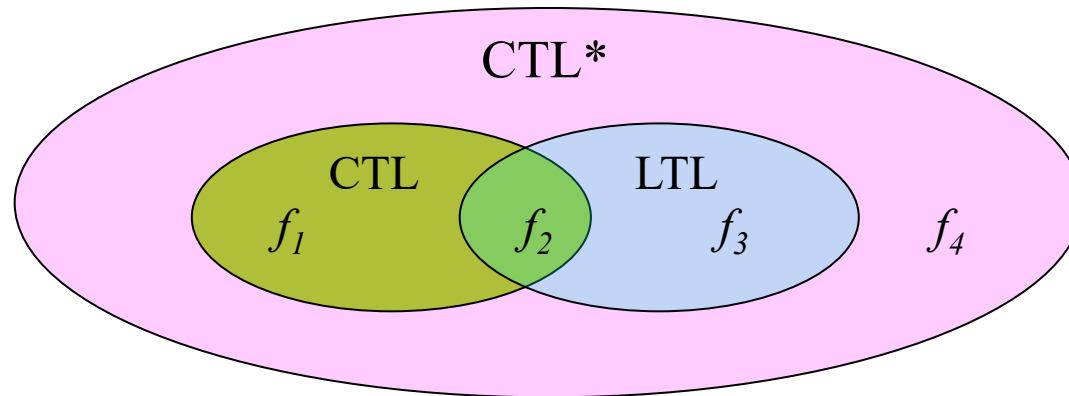
$$LTL \cap CTL \neq \emptyset$$

- In LTL and CTL
 - $f_2 = AG(p \rightarrow AF q) \in CTL \Leftrightarrow G(p \rightarrow F q) \in LTL$
 - Any p is eventually followed by a q .



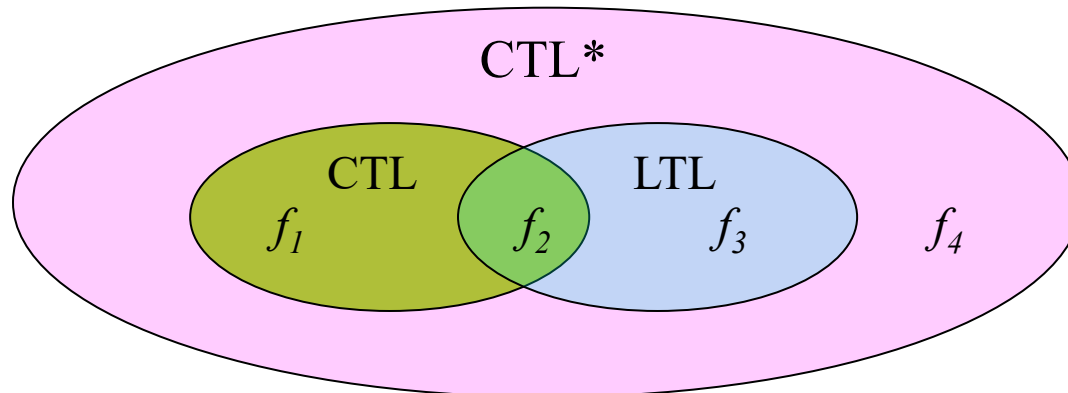
LTL-CTL $\neq\emptyset$

- In LTL but not in CTL (Many fairness constraints)
 - $f_3 = GF\ p \rightarrow F\ q$
 - If there are infinitely many p along the path, then there is an occurrence of f_3 .

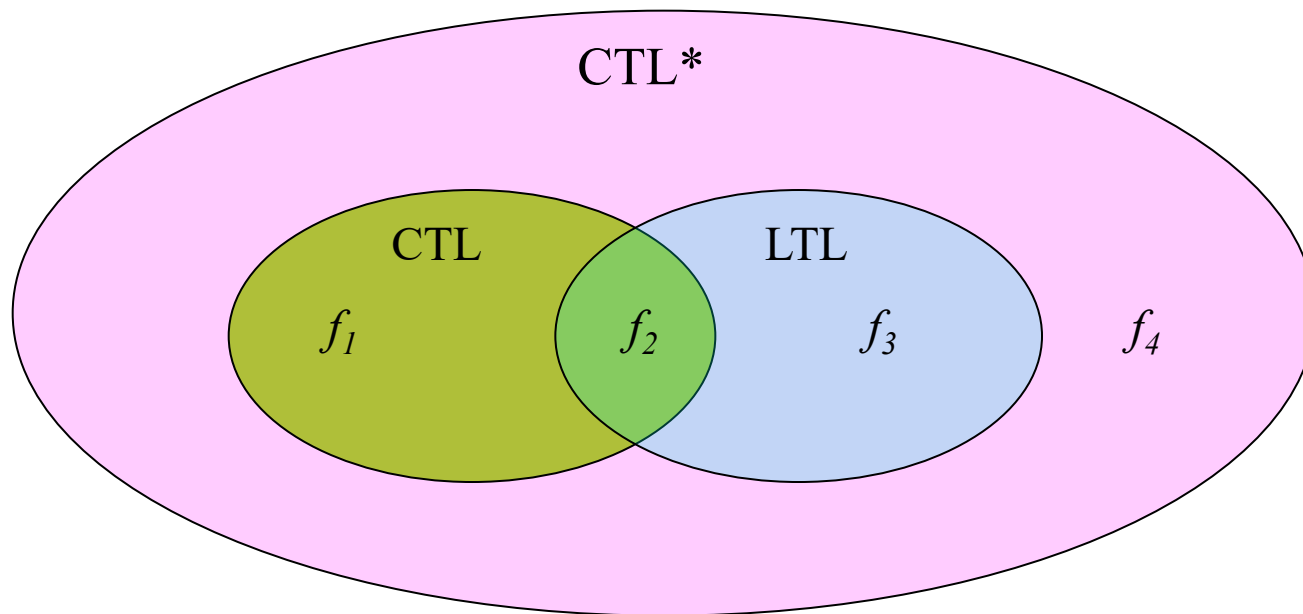


$$\text{CTL}^* - (\text{CTL} \cup \text{LTL}) \neq \emptyset$$

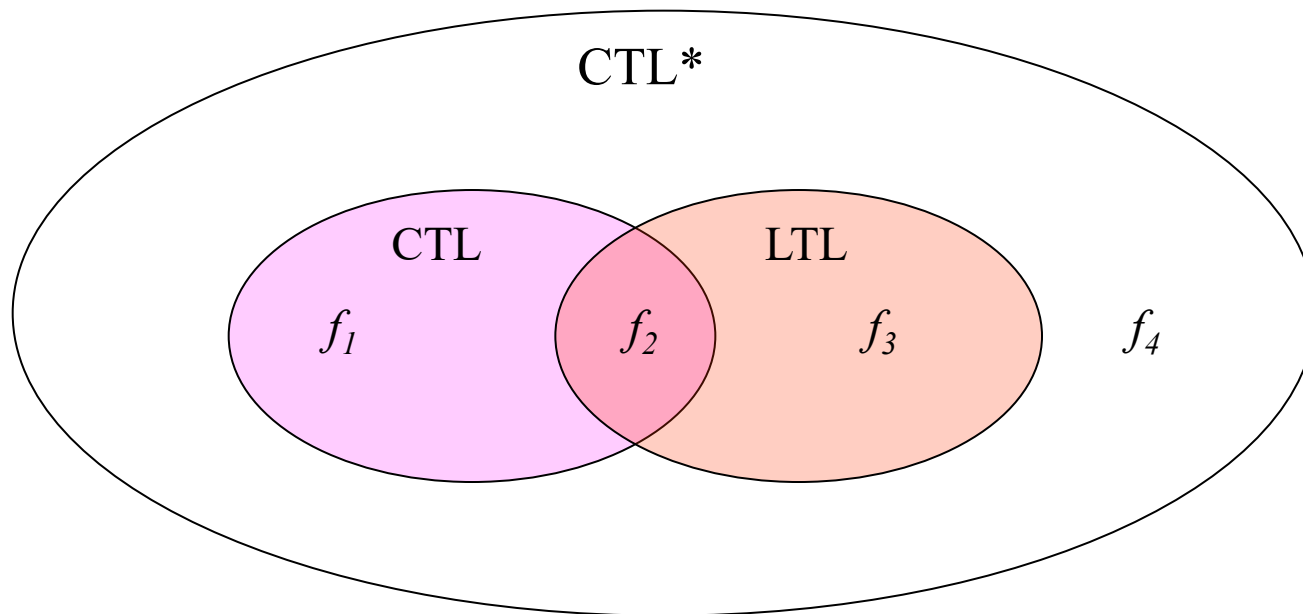
- In CTL^* , but neither in CTL nor in LTL
 - $f_4 = E[GF\ p]$
 - There is a path with infinitely many p .



LTL and CTL as Subsets of CTL*



$$\text{PSL} \supset \text{LTL} \cup \text{CTL}$$



References

- E. A. Emerson. Temporal Logics in Handbook of Theoretical Computer Science. Elsevier Science Publishers B.V., 1990
- Z. Manna, A. Pnueli. The Temporal Logic of Rective and Concurrent Systems. Springer-Verlag, 1991.
- A. P. Sistla, E. M. Clarke. The complexity of propositional linear temporal logics. JACM, 32(3), 1985, pp. 733-749.
- E. A. Emerson, E.M. Clarke. "Sometimes" and "Not Never" revisited: on branching versus linear time temporal logic. JACM, 33(1), 1986, pp. 151-178.
- Principles of Model Checking by Christel Baier and Joost-Pieter Katoen. The MIT Press; 2008.
- Model checking, by E. M. Clarke, etc. The MIT Press; second edition, 2018.
- M. Huth, M. Ryan, Logic in Computer Science: Modeling and Reasoning about Systems, Cambridge University Press; second edition, 2004.